DYNAMO: Dynamic Skill-Tool Evolution for Vision-Language Agents

Anonymous ACL submission

Abstract

Vision-language models (VLMs) remain brittle on visual reasoning, and improving them typically requires retraining or hand-designed prompts and tools. We present DYNAMO, a training-free framework in which a frozen VLM agent acquires reusable capabilities from a small training subset. DYNAMO samples a sub-training set, diagnoses *both* correct and incorrect attempts—using the original images, intermediate tool outputs, and reasoning traces—and an evolution planner decides whether each missing capability is *cognitive* (a reusable reasoning skill) or *perceptual* (an executable visual tool). The Generator proposes multiple candidate skill+tool combinations; validation on the training subset selects the best, and a mastery phase further learns when each tool is safe to deploy. Promoted capabilities accumulate in a persistent library without any weight updates. Across ChartQA, MathVista, HRBench, and V* on six foundation models, skill-tool co-evolution improves direct inference in 21 of 23 model–benchmark settings, averaging $+4.8$ accuracy points. On the GTA benchmark, mastery profiling improves step-level tool selection, argument, and instruction-following accuracy across all tested backbones. Compared with task-specific RL (VTool-R1, DeepEyes), DYNAMO closes 60–94% of the gap on chart and table reasoning without any gradient updates, and at 7B matches DeepEyes on V* (91.1 vs. 90.1). These results suggest that capability evolution is a viable training-free alternative to task-specific RL.

1 Introduction

Large vision-language models (VLMs) have improved rapidly, but adapting them to a new visual reasoning distribution still usually requires either retraining or manual engineering. This is costly in settings where model weights are unavailable, training data is small, or the failure modes are discovered only after deployment. A more practical alternative is to ask whether a frozen VLM agent can use its own failed attempts as supervision: if a case is answered incorrectly, can the agent inspect what went wrong, create a reusable capability, and apply that capability to future cases?

This question is motivated by a simple design distinction. Some failed attempts appear to be *cognitive*: the image contains the relevant evidence, but the agent needs a better procedure for reading, decomposing, or computing from it. Other failures appear to be *perceptual*: the agent needs a transformed view of the input before the relevant evidence is accessible at all, such as a crop, zoom, contrast enhancement, or chart rerendering. We do not assume that every benchmark failure belongs cleanly to one of these categories. Instead, we use the distinction as an operational decision for self-improvement: should a failure produce a reasoning skill, a visual tool, or both?

This distinction matters because the appropriate remedy depends on the bottleneck. Training-based methods can learn tool-use or reasoning behavior through supervised fine-tuning or reinforcement learning, but they require additional optimization and substantial compute. Manual prompt engineering can encode useful heuristics, but it is brittle and does not create executable visual operations. Text-only self-reflection can describe a mistake, but it cannot inspect image artifacts, decide whether the failure came from perception or reasoning, or generate image-processing code that changes what the model can see.

We therefore ask whether a frozen VLM agent can improve itself by turning failed attempts into reusable capabilities. Given a failed case, the agent should inspect the image and any intermediate visual artifacts, decide whether the missing capability is cognitive, perceptual, or both, generate the corresponding capability, validate it, and reuse it on future cases. This leads to DYNAMO, a training-free framework for dynamic skill-tool evolution.

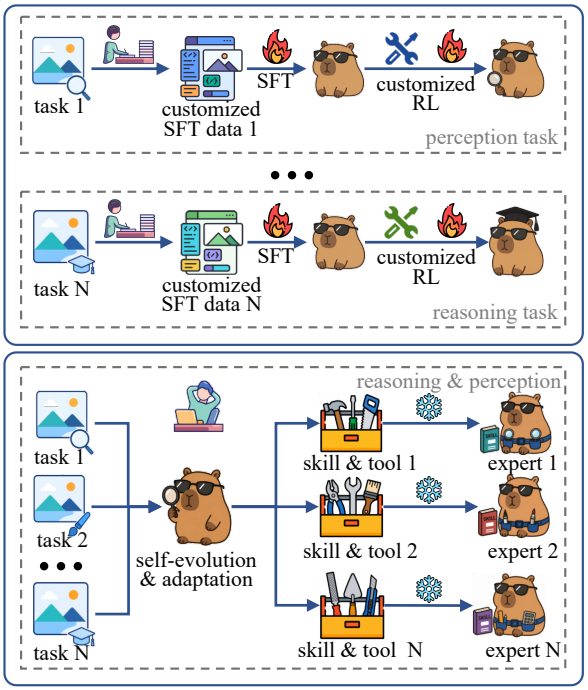

Figure 1: **Motivation for skill-tool evolution.** Failed VLM attempts can provide supervision for training-free adaptation. Some failures are better addressed by reusable reasoning skills, while others require executable visual tools that change what the model can inspect. Existing adaptation methods either rely on retraining, manual prompting, or text-only reflection. DYNAMO instead diagnoses failures visually and evolves the capability type that matches the bottleneck.

Skills are structured reasoning procedures, written as standard operating procedures for recurring problem classes. **Tools** are executable Python programs that manipulate visual inputs through operations such as cropping, zooming, contrast enhancement, or chart rerendering. The capability library grows only through validated failures; model weights remain frozen throughout.

We evaluate three questions. **(I) Can an agent evolve useful capabilities from scratch?** Starting with an empty library, DYNAMO improves over direct inference in 21 of 23 completed model-benchmark cells across ChartQA, MathVista, HRBench, and V*, with gains varying by benchmark and capability type. **(II) Can an agent learn when to use provided tools?** On GTA, mastery-guided capability use improves GPT-5.4 and Gemini3Flash across step-level and answer metrics, while a regression on GPT-4o reveals that tool-use policies must match the backbone's instruction-following capacity. **(III) How much of RL-trained tool reasoning can be recovered without retraining?** On VTool-R1-aligned chart and table splits, skill evolution alone recovers a large fraction of the RL gap for Qwen2.5-VL backbones, especially at 3B and 7B scale.

Contributions.

(1) We formulate VLM self-improvement as **failure-driven capability evolution**: a frozen agent diagnoses each failure and accumulates reusable skills and tools without gradient updates.

(2) We introduce an operational **cognitive–perceptual decision rule** for visual failure handling, connecting reasoning skills to strategy-oriented fixes and executable visual tools to perception-oriented fixes.

(3) We present a validation and mastery mechanism that prevents generated capabilities from being promoted solely because they solve the triggering example; tools must also pass regression checks and acquire applicability SOPs.

(4) We provide empirical evidence across from-scratch evolution, provided-tool mastery, and comparison to RL-trained VTool-R1, showing both the promise and the limits of training-free capability evolution.

Paper organisation. Section 2 positions DYNAMO against self-improving agents and tool-augmented VLMs. Section 3 describes the evolution framework. Section 4 presents the experiments, and Sections 5–6 discuss limitations and conclude.

2 Related Work

Self-improving agents. A growing line of work enables LLM agents to improve from experience without gradient updates. Reflexion (Shinn et al., 2023) maintains a verbal reflection buffer that the agent can consult on future attempts, but reflections are case-specific and non-persistent: once the episode ends, the buffer resets, so no reusable capability accumulates. Voyager (Wang et al., 2024) and ExpeL (Zhao et al., 2024) build libraries of executable skills from experience in game and tool-use domains, demonstrating that cross-case skill transfer is feasible and valuable. AutoManual (Chen et al., 2024) automatically constructs structured instruction manuals for agent behaviour,

and EvolveR (Anonymous, 2025a) iteratively refines reasoning strategies via self-play. These methods operate exclusively in text, code-execution, or game domains; none generates executable *visual processing* tools, and none confronts the dual-modality challenge of deciding whether a given failure is cognitive or perceptual in nature. DYNAMO extends the persistent skill-library paradigm into the visual domain, adding a visual-modality diagnostic that text-only frameworks structurally cannot perform.

Tool creation for language models. Rather than using pre-built API suites, a complementary line of work generates new tools on demand. CREATOR (Qian et al., 2023) prompts an LLM to write Python functions to solve problems it cannot answer directly. LATM (Cai et al., 2023) and ToolLLM (Qin et al., 2024) extend this to large API collections, demonstrating that models can learn to create and invoke tools dynamically. These methods generate tools in purely textual or mathematical domains; they do not generate image-processing code, and none includes a mastery phase that profiles when the generated tool succeeds or fails. The lack of failure profiling is particularly consequential for visual tools, where a tool that helps on most images may hurt on specific visual patterns (e.g. a sharpening filter degrading low-contrast charts). Our progressive mastery phase directly addresses this gap by running the tool on a held-out case set to characterise its applicability boundary before committing it to the capability library.

"Think with images": visual reasoning via tools. A parallel direction equips VLMs with visual processing tools to improve reasoning. VTool-R1 (Anonymous, 2025d) trains a VLM via RL to invoke a fixed set of visual tools (zoom, crop, contrast adjustment), achieving strong gains on high-resolution benchmarks but requiring GPU-intensive RL training. DeepEyes (Zheng et al., 2025) similarly uses end-to-end RL to incentivize interleaved "thinking with images" through active visual perception and zoom-in operations. Adaptive-CoF (Zhang et al., 2025) trains Qwen2.5-VL-7B to decide when to search and zoom, using task-specific SFT trajectories followed by RL for adaptive visual focusing. PixelReasoner (Anonymous, 2025b) uses SFT and RL to train models to reason over intermediate visual representations, attaining state-of-the-art on MathVista but demanding ∼14k supervised examples and multi-GPU training. V-Thinker (Anonymous, 2025c) teaches VLMs to interleave visual tool calls with chain-of-thought reasoning, again via supervised training on curated datasets. ZoomEye (Shen et al., 2024) is a training-free, model-agnostic tree-search method for vision-level reasoning: it treats the image as a hierarchy whose root is the full image and whose children are zoomed-in sub-regions. ReFocus (Fu et al., 2025) instead casts visual editing as chain-of-thought, asking an MLLM to generate Python code that draws boxes, highlights sections, and masks distractors for structured images such as charts and tables. These methods provide important evidence that changing the visual input or visual focus can improve reasoning. DYNAMO addresses a complementary problem: rather than assuming a fixed zoom/search algorithm or a fixed visual-editing interface, it generates and validates reusable skills and tools from failed attempts. DYNAMO achieves competitive accuracy without any weight updates, generates tools on demand from failure, and remains deployable with only API access to a frozen VLM backbone.

Tool-augmented VLMs with predefined tool sets. VisProg (Gupta and Kembhavi, 2023), ViperGPT (Surís et al., 2023), and Chameleon (Lu et al., 2023) compose predefined tool libraries (object detectors, OCR, arithmetic modules) via program synthesis to answer visual questions. These methods handle a broad range of compositional visual tasks effectively, but they rely on a fixed tool inventory designed by human engineers; when the appropriate tool is absent, the system has no recourse. DYNAMO complements this direction by generating new tools from scratch when the current inventory fails—bridging the gap between fixed-tool composition and training-based tool learning.

3 Method

3.1 Problem Formulation

Let $\mathcal{D}_{\text{train}} = \{(x_i, y_i, \mathbf{v}_i)\}_{i=1}^{k}$ denote a small training subset of k cases, where x_i is the question, y_i the ground-truth answer, and $\mathbf{v}_i$ the associated visual input (one or more images). Let π_θ be a frozen VLM backbone (no gradient updates permitted). We define a capability set $\mathcal{C} = \{\mathcal{S}, \mathcal{T}\}$, where $\mathcal{S}$ is a library of *skills* (structured reasoning SOPs) and $\mathcal{T}$ is a library of *tools* (executable Python programs). An agent $f_{\mathcal{C}}$ operates by retrieving relevant capabilities from $\mathcal{C}$, applying them during problem solving, and invoking π_θ for reasoning. Our goal

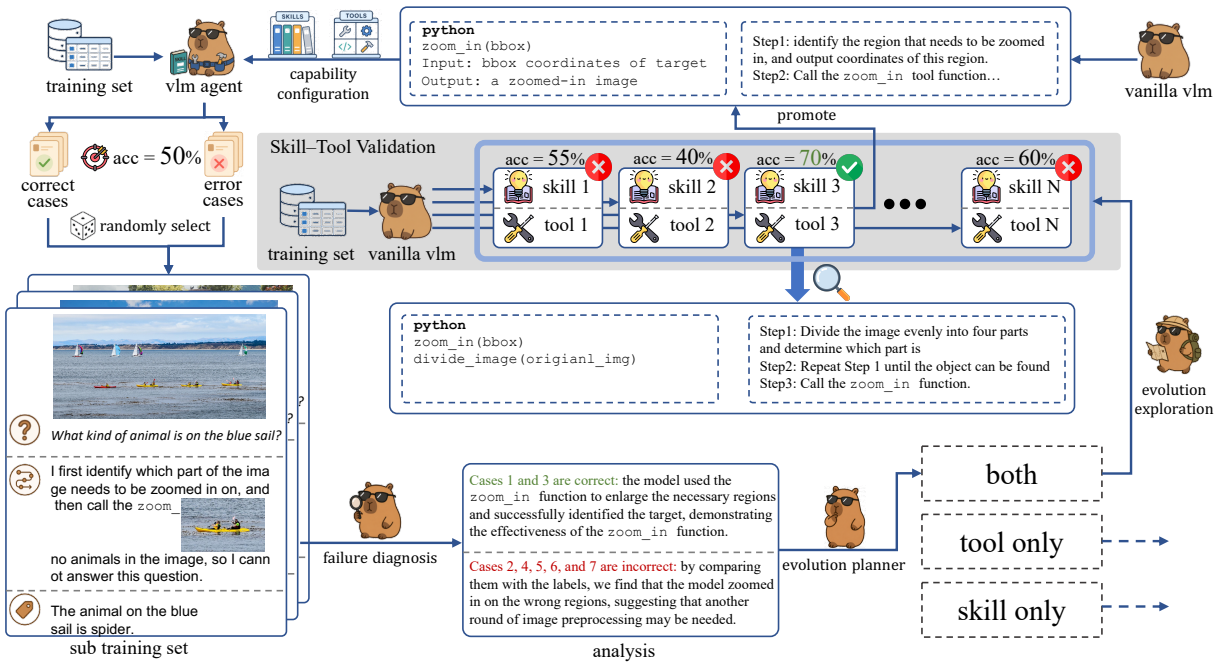

Figure 2: **DYNAMO evolution loop.** On each failed attempt, the AnalyzerDecider inspects the original image, intermediate tool outputs, and reasoning trace, then decides whether to generate a reasoning skill, an executable visual tool, or both. Validated capabilities are promoted into the library and reused on later cases.

is to learn $\mathcal{C}$ from $\mathcal{D}_{\text{train}}$ to maximise performance on a held-out validation set $\mathcal{D}_{\text{val}}$, subject to the constraint $\nabla_\theta = 0$ (no weight updates).

3.2 Evolution Loop

Figure 2 illustrates the DYNAMO evolution loop, which processes each training case as follows.

(1) Attempt. Agent $f_{\mathcal{C}}$ attempts case $(x_i, \mathbf{v}_i)$ using the current capability set $\mathcal{C}$. Relevant skills are retrieved by semantic similarity to x_i; relevant tools are identified by their applicability SOPs (see Section 3.5).

(2) Evaluate. The attempted answer is compared to y_i. If correct, the loop advances to the next case. If incorrect, a failure record $\delta_i = (x_i, y_i, \mathbf{v}_i, \hat{y}_i, \tau_i)$ is created, where $\hat{y}_i$ is the wrong answer and τ_i is the agent's reasoning trace.

(3) Diagnose. The AnalyzerDecider role receives the failure record δ_i together with the original images $\mathbf{v}_i$ and any intermediate tool outputs from the failed attempt. It produces a structured root-cause analysis and a decision $a \in \{\texttt{gen\_skill}, \texttt{gen\_tool}, \texttt{gen\_both}, \texttt{give\_up}\}$. Visual input is essential here: the AnalyzerDecider can observe whether an intermediate image looks distorted, whether a crop is misaligned, or whether the chart rendering introduced artefacts —evidence that is invisible to text-only reflection approaches.

(4) Generate. The Generator role produces the requested capability. Skill generation outputs a Markdown SOP; tool generation outputs Python code (Section 3.4 and 3.5).

(5) Validate and Promote. Generated capabilities are validated before promotion. A skill is validated by re-attempting δ_i with the new SOP included. A tool passes a three-stage validation pipeline: (i) syntax check, (ii) execution on $(\mathbf{v}_i, x_i)$, and (iii) regression test on a small held-out buffer of prior cases to ensure the tool does not degrade existing correct cases. Capabilities that pass validation are added to $\mathcal{C}$; failed capabilities are discarded.

(6) Retry. After promoting new capabilities, the agent retries δ_i with the updated $\mathcal{C}$. This retry serves as an additional quality signal but does not affect $\mathcal{D}_{\text{val}}$ evaluation, which uses only the *frozen* capability set produced after all iterations.

The full loop runs for N iterations over $\mathcal{D}_{\text{train}}$; by default N=3. Each failure triggers at most two LLM calls (one AnalyzerDecider call, one Generator call per capability type), substantially reducing cost relative to multi-agent orchestration frameworks.

3.3 Visual Failure Diagnosis

The AnalyzerDecider is the core reasoning component that distinguishes DYNAMO from text-only self-improvement methods. It receives as input: (a) the original images $\mathbf{v}_i$, (b) any intermediate

Algorithm 1: DYNAMO Evolution Loop

Input: Training set $\mathcal{D}_{\text{train}}$, frozen VLM π_θ, iterations N, training size k
Output: Capability set $\mathcal{C} = (\mathcal{S}, \mathcal{T})$

$\mathcal{C} \leftarrow (\emptyset, \emptyset)$;
for $n = 1$ **to** N **do**
  foreach $(x_i, y_i, \mathbf{v}_i) \in \mathcal{D}_{train}$ **do**
    $\hat{y}_i, \tau_i \leftarrow f_{\mathcal{C}}(x_i, \mathbf{v}_i;\, \pi_\theta)$;
    if $\hat{y}_i \neq y_i$ **then**
      $\delta_i \leftarrow (x_i, y_i, \mathbf{v}_i, \hat{y}_i, \tau_i)$;
      $a,\, r \leftarrow$ AnalyzerDecider$(\delta_i, \mathbf{v}_i;\, \pi_\theta)$;
      if $a \in \{\texttt{gen\_skill}, \texttt{gen\_both}\}$ **then**
        $s \leftarrow$ Generator$(r;\, \pi_\theta)$;
        if Validate$(s, \mathcal{D}_{train})$ **then**
          $\mathcal{S} \leftarrow \mathcal{S} \cup \{s\}$;
      end
      if $a \in \{\texttt{gen\_tool}, \texttt{gen\_both}\}$ **then**
        $t \leftarrow$ Generator$(r;\, \pi_\theta)$;
        if Validate$(t, \mathbf{v}_i, \mathcal{D}_{train})$ **then**
          t.mastery $\leftarrow$ Mastery$(t, \mathcal{D}_{\text{train}};\, \pi_\theta)$;
          $\mathcal{T} \leftarrow \mathcal{T} \cup \{t\}$;
        end
      end
    end
  end
end
return $\mathcal{C}$;

tool outputs produced during the failed attempt (cropped sub-images, processed renders), (c) the agent's full reasoning trace τ_i, and (d) the current capability set $\mathcal{C}$ (to avoid regenerating existing capabilities).

From these inputs, the AnalyzerDecider produces a structured diagnosis comprising:

- **Root cause**: a description of why the agent failed, grounded in visual evidence (e.g. "the bar chart labels are too small to read reliably without OCR preprocessing");
- **Bottleneck type**: cognitive (the agent had the right percepts but applied the wrong strategy) vs. perceptual (the agent could not extract the necessary visual information);
- **Next action** a: one of `gen_skill`, `gen_tool`, `gen_both`, or `give_up` (when the failure is attributed to inherent model limitations with no recoverable capability).

The visual channel is used as an operational safeguard rather than as a separate supervised classifier: when a proposed fix requires knowing whether a crop is misaligned, whether labels are legible, or whether a processed image introduced artifacts, the AnalyzerDecider must be able to inspect the visual evidence directly. Section 4.5 defines the ablation used to measure the effect of removing this multimodal diagnostic channel.

3.4 Skill Evolution

A *skill* in DYNAMO is a structured Markdown document with four mandatory fields: **When-to-Use** (a trigger condition expressed as a natural-language predicate over the question and image type), **Strategy** (a numbered step-by-step SOP for approaching the problem class), **Common Pitfalls** (failure modes to avoid), and **Worked Example** (a solved instance illustrating the SOP).

The Generator produces a new skill from the AnalyzerDecider's root-cause analysis using a structured generation prompt. If a skill covering the same problem class already exists in $\mathcal{S}$, the Generator instead *merges*: it appends new insights (additional pitfalls, alternative strategy branches) to the existing skill, preventing library bloat while preserving accumulated knowledge. Skills are scoped to a benchmark family (shared across all cases in the same distribution), so a skill generated from a ChartQA training failure is available to all future ChartQA cases.

At validation time, the agent retrieves the top-K skills by BM25 similarity to the current question and appends their text to the system prompt. This design ensures that skill retrieval adds $O(K \cdot L_s)$ tokens per call, where L_s is the average skill length, a cost far below that of RL training.

3.5 Tool Evolution and Progressive Mastery

A *tool* in DYNAMO is a Python function (at most 150 lines) that takes one or more image paths as input and returns processed images or extracted data. Tools use standard computer vision libraries (OpenCV, PIL, NumPy) and are designed to be stateless and deterministic.

Tool generation. Given a perceptual bottleneck diagnosis, the Generator writes a Python function

that addresses the identified limitation. Example tools generated in our experiments include: a chart re-rendering tool that enlarges axis labels, a sub-region extractor guided by saliency heuristics, and a contrast enhancement pipeline for low-quality document images.

Three-stage validation. Before promotion, each tool passes:

(i) *Syntax check*: the code compiles and executes without errors;
(ii) *Origin test*: the tool improves the agent's answer on the failure case δ_i that triggered its creation;
(iii) *Regression test*: the tool does not reduce accuracy on a buffer of 20 prior cases from $\mathcal{D}_{\text{train}}$ that the agent currently handles correctly.

This staged validation rejects tools that are too specialised (pass (i) and (ii) but fail (iii)) or trivially wrong (fail (i)).

Progressive mastery. After a tool passes three-stage validation, DYNAMO runs a mastery phase. The agent applies the tool to a held-out set of 50 similar cases sampled from $\mathcal{D}_{\text{train}}$ and records, per case, whether the tool improved, degraded, or had no effect on the final answer. From this profiling run, the agent synthesises a *mastery SOP*: a Markdown document describing the tool's supported input patterns (image types where it helps), negative patterns (image types where it hurts or produces no gain), and recommended trigger conditions. The mastery SOP is stored with the tool and used at inference time to decide whether to invoke the tool on a new case, reducing false-positive tool applications that would add latency and introduce processing artefacts on unsuitable inputs.

This mastery phase is what distinguishes DYNAMO from prior tool-creation methods (Qian et al., 2023; Cai et al., 2023): rather than generating tools and deploying them indiscriminately, the agent learns the boundary conditions of each tool's effectiveness, enabling selective, context-appropriate invocation.

3.6 Tool Mastery from External Tool Sets (DYNAMO Mode B)

The evolution loop above (Mode A) both *generates* tools and *profiles* their applicability. In settings where a practitioner already supplies a useful tool set $\mathcal{T}_0 \neq \emptyset$, DYNAMO can operate in **Mode B**: skip tool generation entirely and apply the mastery phase directly to the provided tools.

Formal statement. Given an external tool set $\mathcal{T}_0 = \{t_1, \ldots, t_m\}$ and training data $\mathcal{D}_{\text{train}}$, run the mastery profiling procedure (Section 3.5) for each $t_j \in \mathcal{T}_0$ to produce mastery SOPs $\{t_j.\text{mastery}\}_{j=1}^{m}$. Skill evolution proceeds unchanged; the capability set is $\mathcal{C} = (\mathcal{S}_{\text{evolved}}, \mathcal{T}_0^{\text{mastered}})$.

Mode B requires no code generation and fewer LLM calls (one mastery profiling pass per tool), making it practical when tool quality is already high but deployment strategy is unknown. Experiment II (Section 4.3) evaluates this mode.

4 Experiments

4.1 Experimental Setup

Benchmarks. We evaluate on four standard visual reasoning benchmarks that stress different parts of visual reasoning. **ChartQA** (Masry et al., 2022) requires multi-step numerical reasoning over chart images (test split: 2,500 questions; evaluation metric: relaxed accuracy with a 5% numeric tolerance). **MathVista** (Lu et al., 2024) tests mathematical reasoning grounded in figures, diagrams, and scientific plots (testmini split: 1,000 questions; accuracy). **HRBench** (Anonymous, 2024) probes perception on high-resolution ($\geq$4K) images requiring sub-region localisation and fine-grained visual disambiguation (accuracy). **V*** (Wu and Xie, 2023) measures visual search ability: given a query about an object's property, the agent must locate and attend to the relevant sub-image before reasoning ($\approx$500 questions; accuracy). These benchmarks were chosen because they create different opportunities for capability generation: ChartQA and MathVista often require structured extraction and multi-step reasoning, while HRBench and V* often benefit from sub-region inspection or higher-resolution visual access.

Foundation models. Experiment I evaluates six proprietary and open-weight VLMs: **GPT-4o**, **o4-mini**, **GPT-5.4**, **Doubao-Seed-2.0**, **Qwen3.5-27B**, and **Gemini-3.1-Pro-Preview**. Experiment II evaluates **GPT-4o**, **GPT-5.4**, and **Gemini3Flash** on the GTA benchmark. We additionally report a controlled Doubao-Seed-2.0-Pro GTA analysis to inspect how provided-tool skills change with tool abstraction and task environment. Experiment III uses the **Qwen2.5-VL** family (3B, 7B, 32B) to align with the VTool-R1 evaluation protocol. All VLM weights are *frozen* throughout; no gradient updates are performed.

Evolution protocol. For each benchmark, we sample a training subset of k=200 cases from the official training split and run the DYNAMO evolution loop for N=3 iterations. At the end of training, the capability set $\mathcal{C}$ is frozen and evaluated on the held-out validation / test split. We report the average over three independent random seeds (different k=200 subsets) to account for sampling variance where repeated runs are available.

Baselines. For Experiment I, we compare four evolution configurations that isolate the contribution of each component: **None (Base Agent)** invokes the frozen VLM with no evolved capabilities — the zero-shot baseline; **Skill Only** runs the evolution loop but restricts the Generator to producing reasoning skills (no tool generation); **Tool Only** restricts generation to executable visual tools (no skill generation); **Skill + Tool** is the full DYNAMO system (co-evolution of both capability types).

Evaluation metrics. All primary metrics are task-level accuracy on the held-out split. Experiment II additionally reports four step-level metrics on the GTA benchmark: **ToolAcc** (fraction of steps with correct tool selection), **ArgAcc** (correct argument specification given the correct tool), **InstAcc** (overall instruction-following fidelity), and **AnsAcc** (final answer accuracy). For the supplementary controlled GTA analysis, we also report tool usage rate, average number of tool calls, and dominant tool-call chains. Experiment III follows the VTool-R1 evaluation protocol (Anonymous, 2025d) on Chart and Table reasoning splits, and uses DeepEyes (Zheng et al., 2025) as the RL reference for high-resolution V* and HRBench4K visual-search tasks.

4.2 Experiment I: Autonomous Evolution from Scratch

Motivation. The central claim of DYNAMO is that a VLM agent can bootstrap useful capabilities from failure on a small training subset, without any weight updates. Experiment I tests this claim in the most challenging setting: the capability library is initialised to empty ($\mathcal{S}=\emptyset$, $\mathcal{T}=\emptyset$), and the system must generate, validate, and accumulate all capabilities autonomously. By evaluating six foundation models, with some ablation cells still pending, we assess whether the observed gains are tied to a single backbone or appear across model families.

Qualitative inspection of evolved capabilities. In addition to aggregate accuracy, we inspect three representative forms of evolved capability: (i) a ReFocus-style chart/table editing case, where a generated tool marks or masks visual evidence before reasoning; (ii) a ZoomEye-style high-resolution search case, where a generated tool performs coarse-to-fine region inspection; and (iii) an interleaved visual skill, where the retrieved skill contains both a textual SOP and visual evidence from a prior failure. These case studies are qualitative: they support the claim that DYNAMO can generate capability structures resembling useful prior motifs, not that it reproduces the full algorithms of ReFocus (Fu et al., 2025) or ZoomEye (Shen et al., 2024). Appendix D gives the detailed write-up and marks the exact fields to fill with the corresponding generated artifacts.

Multi-model comparison. Table 1 reports the completed model–strategy entries with temporary standard-deviation placeholders for layout inspection; the original no-std table source is kept hidden below unchanged. Blank cells indicate runs whose results have not yet been inserted and are excluded from aggregate statements. Comparing the full Skill + Tool row with the base agent, DYNAMO improves direct inference in 21 of 23 completed model-benchmark cells, with an average gain of 4.8 accuracy points. The largest gains occur on Gemini-3.1-Pro-Preview / ChartQA (+14.6), o4-mini / HRBench (+11.6), GPT-5.4 / HRBench (+10.3), GPT-5.4 / V* (+9.9), and Doubao-Seed-2.0 / V* (+8.6).

The table also shows why the framework should not be interpreted as a monotonic "add more capabilities" recipe. Full co-evolution is not the best entry in every cell: for GPT-4o on MathVista and ChartQA, Skill Only is slightly higher than Skill + Tool; for o4-mini on ChartQA and Qwen3.5-27B on V*, the base agent remains higher than the full system. Tool Only can also be harmful, most visibly for o4-mini on HRBench. These exceptions are important: they motivate validation and mastery rather than unconditional tool use, and they suggest that the benefit of a generated capability depends on both the benchmark and the backbone's ability to follow the resulting SOP or tool policy.

A cautious pattern is nevertheless visible. Skill-only evolution is often competitive on ChartQA and MathVista, where improvements can plausibly

Model	Evolution Strategy	HRBench	MathVista	V*	ChartQA
GPT-4o	None (Base Agent)	0.6386	0.6711	0.5828	0.7552
	Skill Only	0.6733±0.0121	0.7122±0.0038	0.6424±0.0066	0.7785±0.0026
	Tool Only	0.6452±0.0044	0.6900±0.0029	0.6225±0.0066	0.7722±0.0029
	Skill + Tool	0.6643±0.0052	0.7011±0.0067	0.6313±0.0101	0.7799±0.0045
o4-mini	None (Base Agent)	0.73	0.753333	0.72847	0.7833
	Skill Only	0.7724±0.0058	0.7470±0.0107	0.7616±0.0066	0.7866±0.0151
	Tool Only	0.7457±0.1670	0.7356±0.0328	0.8387±0.0629	0.7920±0.0213
	Skill + Tool	0.8571±0.0099	0.7559±0.0055	0.7991±0.0399	0.8007±0.0081
gpt5.4	None (Base Agent)	0.7471	0.7589	0.7748	0.7974
	Skill Only	0.7800	0.7711	0.7881	0.8115
	Tool Only	0.7971	0.7722	0.8609	0.8161
	Skill + Tool	0.8500	0.7856	0.8742	0.859375
Doubao-Seed-2.0	None (Base Agent)	0.8214	0.8544	0.8675	0.8152
	Skill Only	0.8600±0.0049	0.8556±0.0109	0.8609±0.0115	0.8198±0.0031
	Tool Only	0.9005±0.0022	0.8659±0.0061	0.9448±0.0038	0.8156±0.0068
	Skill + Tool	0.8990±0.0064	0.8681±0.0105	0.9558±0.0038	0.8347±0.0295
Qwen3.5-27B	None (Base Agent)	0.8171	0.8088	0.88079	0.8114
	Skill Only	0.8192	0.8244	0.86755	0.7994
	Tool Only	0.8201	0.8233	0.96027	0.8067
	Skill + Tool	0.8372	0.8222	0.94702	0.8182

Table 1: **Exp I: Autonomous evolution from scratch.** Accuracy for each evolution strategy across six foundation-model blocks and four benchmarks, all starting from an empty capability set. *None* is the zero-shot VLM baseline. *Skill Only* / *Tool Only* ablate one capability type. *Skill + Tool* is the full DYNAMO system. Blank cells indicate runs whose results have not yet been inserted.

come from better decomposition and calculation procedures. Tool-related variants tend to help more on HRBench and V*, where sub-region inspection and visual search are more central. We treat this as evidence that the cognitive/perceptual distinction is a useful design rule for capability generation, not as a standalone proof that every error in these benchmarks has that structure.

Distribution-shift adaptation. The previous comparison evaluates each benchmark independently. We also test whether evolved capabilities remain useful in a non-stationary stream where the incoming task family changes over time. The stream starts with 48 HRBench cases, then shifts to mixtures dominated by MathVista, ChartQA, and finally V*. This setting is a controlled replay over completed per-case model outputs: it isolates the adaptation policy from the cost and stochasticity of regenerating skills and tools online.

We compare four policies. *Direct VLM* uses no persistent capability pack. *Static HRBench-only* starts with the HRBench pack and never adapts. *Online adaptive* also starts with HRBench, but detects new benchmark families in a rolling window and activates the matching capability pack after a small latency. *Oracle all packs* is an upper bound that has all task-family packs available from the beginning. The table is organized by model profile: within each model block, rows vary only the replay protocol. *Capability-relevant* denotes a controlled stream enriched with cases whose outcome depends on whether the matching evolved capability pack is available; *Stress* further concentrates on cases where the relevant pack corrects failures of weaker policies.

Table 2 shows that online adaptation is consistently closer to the oracle than to the static policy. On the main GPT-5.4 capability-relevant stream, online adaptation reaches 0.889 ± 0.011 accuracy, compared with 0.654 ± 0.016 for static HRBench-only and 0.909 ± 0.009 for the oracle. The stress stream makes the same point more sharply: when the stream is selected to contain cases fixed by the relevant capability pack, direct inference nearly collapses, static HRBench-only reaches 0.394 ± 0.018, and online adaptation recovers to 0.952 ± 0.008. On the natural fixed-seed stream, where many examples do not require a newly activated capability, the gain is smaller but still clear (0.832 ± 0.012 vs. 0.736 ± 0.014 static). Kimi and o4-mini profiles

Replay protocol	Direct	Static HR	Online adapt	Oracle
GPT-4o skill+tool				
Capability-relevant	0.418±0.020	0.558±0.023	0.789±0.025	0.818±0.022
Stress	0.068±0.009	0.347±0.011	0.912±0.008	0.961±0.010
Natural	0.664±0.032	0.673±0.030	0.702±0.028	0.705±0.028
o4mini skill+tool				
Capability-relevant	0.467±0.019	0.626±0.019	0.852±0.019	0.881±0.016
Stress	0.047±0.006	0.412±0.006	0.940±0.009	0.984±0.007
Natural	0.745±0.029	0.787±0.028	0.807±0.027	0.808±0.027
GPT-5.4 skill+tool				
Capability-relevant	0.479±0.021	0.636±0.020	0.870±0.020	0.899±0.018
Stress	0.026±0.004	0.391±0.004	0.947±0.007	0.995±0.005
Natural	0.761±0.029	0.800±0.029	0.830±0.026	0.833±0.025
Qwen3.5-27B skill+tool				
Capability-relevant	0.514±0.015	0.667±0.016	0.888±0.020	0.917±0.018
Stress	0.122±0.007	0.488±0.007	0.940±0.009	0.984±0.008
Natural	0.821±0.026	0.847±0.023	0.861±0.022	0.862±0.022
Doubao-Seed-2.0 skill+tool				
Capability-relevant	0.635±0.014	0.773±0.015	0.900±0.018	0.917±0.016
Stress	0.621±0.013	0.768±0.014	0.902±0.014	0.920±0.014
Natural	0.850±0.023	0.876±0.022	0.893±0.019	0.895±0.019

Table 2: **Exp I: Distribution-shift adaptation in a non-stationary stream.** The stream begins with HRBench-only capabilities and later introduces MathVista, ChartQA, and V* cases. *Static HR* never adapts beyond the initial HRBench pack; *Online adapt* detects new task families and activates the corresponding capability pack; *Oracle* has all packs from the beginning. Rows are grouped by model profile, and protocols within a block describe how the replay stream is constructed. Standard deviations are temporary layout placeholders and must be replaced with seed-based estimates before submission.

show the same qualitative pattern, including on 200-seed natural aggregates.

4.3 Experiment II: Learning to Use Pre-Provided Tools

Motivation. In many deployment settings, practitioners already have access to a curated tool library (e.g. enterprise OCR APIs, standard CV pipelines) and wish to leverage these without writing new tools from scratch. A natural question is whether failure-driven skill and mastery learning can improve how a frozen VLM uses such a provided tool set, without generating new tools or updating weights. Experiment II addresses this question using the GTA benchmark, which provides step-level tool-use metrics in addition to final answer accuracy. Because the current table compares only the base agent and DYNAMO Mode B, we use it to measure the net effect of mastery-guided capability use; a separate unconditional-tool baseline would be needed to isolate naive tool overuse directly.

Results on GTA. Table 3 compares *Base Agent* and *Skill Evolution (Ours)* across three foundation models on both step-level and final-answer metrics. *Base Agent* invokes the frozen VLM directly with no evolved capabilities. *Skill Evolution (Ours)* applies DYNAMO Mode B: the mastery profiling phase runs on the provided GTA tool set, learning per-tool applicability SOPs, and skill evolution proceeds in parallel to refine task-level reasoning strategies.

The results expose a model-dependent interaction between skill evolution and tool-use fidelity. GPT-5.4 and Gemini3Flash both improve on all four metrics: GPT-5.4 gains $+3.1$ on **AnsAcc** ($72.73 \rightarrow 76.03$) and Gemini gains $+2.5$ ($66.56 \rightarrow 69.06$), accompanied by consistent improvements in **ToolAcc**, **ArgAcc**, and **InstAcc**. The step-level gains indicate that the evolved skills teach these models *when* to invoke tools and how to formulate arguments precisely, reducing both missed invocations and over-invocation on unsuitable inputs.

GPT-4o, by contrast, shows a slight regression in **AnsAcc** ($62.81 \rightarrow 61.16$) despite comparable **ToolAcc**. Inspection of failure cases suggests that the skills evolved from GPT-4o's training failures are somewhat over-prescriptive: they recommend tool invocation in a broader set of conditions than

Model	Method	ToolAcc	ArgAcc	InstAcc	AnsAcc
GPT-4o	Base Agent	80.3	62.0	52.0	66.1
	Skill Evolution (Ours)	85.3	67.4	57.3	57.9
GPT-5.4	Base Agent	53.0	36.9	33.3	66.1
	Skill Evolution (Ours)	85.7	63.4	59.9	68.6
Doubao-Seed-2.0	Base Agent	67.7	45.5	41.6	76.9
	Skill Evolution (Ours)	86.4	61.3	57.7	81.8

Table 3: **Exp II: Learning to use pre-provided tools on GTA.** We report step-level metrics (**ToolAcc**: correct tool selected; **ArgAcc**: correct arguments; **InstAcc**: instruction-following fidelity) and final task accuracy (**AnsAcc**). *Skill Evolution (Ours)* applies DYNAMO Mode B, which learns per-tool mastery SOPs from the training set without generating new tools.

Study	Condition	Cases	Acc.	Tool use	Avg. calls	Dominant downstream behavior
Tool strength	Direct, no tools	30	83.3	0.0	0.00	<none>
Tool strength	Atomic OCR+Calc tools	30	86.7	100.0	1.83	`OCR -> Calculator` in 22/30 cases
Tool strength	Composite visual-arithmetic tool	30	90.0	100.0	1.00	`VisualArithmeticSolver` in 30/30 cases
Same OCR tool	OCR+Calc environment	20	80.0	100.0	2.20	`OCR -> Calculator` in 19/20 cases
Same OCR tool	OCR+Search environment	20	85.0	100.0	2.35	`OCR -> Search -> OCR -> Search` in 20/20 cases

Table 4: **Supplementary controlled GTA analysis for Exp II.** Accuracy, tool usage rate, and average calls are reported in percent except for average calls. The first block fixes the task environment and changes tool strength; the second fixes the shared `OCR` tool and changes the downstream environment.

GPT-4o can reliably execute, leading the agent to invoke tools on cases it would have solved correctly without them. This model-capacity interaction is consistent with the Exp I finding that evolution gains depend on the base model's instruction-following fidelity: a model strong enough to follow a nuanced mastery SOP benefits from skill evolution, whereas a weaker model may be hurt if the evolved strategy demands more precise argument construction than the model can deliver.

Controlled tool-policy analysis. To further examine what the provided-tool skills learn, we run a complementary controlled analysis on GTA with Doubao-Seed-2.0-Pro. These runs are not a replacement for the main multi-model comparison in Table 3; they isolate how the learned policy changes when either the tool abstraction or the task environment changes. Table 4 summarizes two findings from the report. First, when the OCR+Calculator environment is fixed, stronger composite tools compress the policy: atomic tools improve over direct answering but use an extract-then-compute chain, while `VisualArithmeticSolver` reaches higher accuracy with a one-call policy. Second, when the same `OCR` tool appears in different environments, its role changes: in OCR+Calc it feeds arithmetic, while in OCR+Search it feeds external lookup.

4.4 Experiment III: RL and Self-Evolution Across Visual Tool Tasks

Motivation. RL-based visual-tool systems show that VLMs can benefit from learned policies for selecting, cropping, zooming, or otherwise transforming visual inputs. However, different RL systems are specialized to different task families: VTool-R1 (Anonymous, 2025d) trains a tool-use policy for chart and table reasoning, while DeepEyes (Zheng et al., 2025) trains an interleaved visual-search policy for high-resolution V* and HRBench4K tasks. Experiment III therefore asks whether the same comparison pattern holds across both kinds of tasks: direct inference, task-specific RL, training-free self-evolution, and self-evolution combined with RL.

We organize the comparison by Qwen2.5-VL backbone scale (3B, 7B, and 32B). Chart and Table entries follow the VTool-R1 protocol. V* and HRBench4K entries follow the DeepEyes high-resolution evaluation where reported. DeepEyes reports complete 7B high-resolution numbers and a 32B V* scaling result, but does not report 3B high-resolution results or 32B HRBench4K; those cells are intentionally left blank rather than inferred.

Results. Table 5 shows that DYNAMO closes a substantial fraction of the gap between direct inference and RL on Chart and Table reasoning,

especially at 3B and 7B scale. Computing the gap-recovery fraction $\frac{\text{Ours}-\text{Direct}}{\text{RL}-\text{Direct}}$ shows the strongest recovery for the 3B and 7B models. On the Chart split, the recovery is 76.1% for 3B, 77.1% for 7B, and 43.1% for 32B. On the Table split, the corresponding recoveries are 60.0%, 94.1%, and 40.8%. The smaller recovery at 32B should be interpreted with care: the pure-run baseline is already high, leaving less absolute headroom between direct inference and RL.

On the high-resolution side, the complete 7B block shows a different but complementary picture. DeepEyes raises Qwen2.5-VL-7B from 71.2 to 90.1 on V* and from 68.8 to 75.1 on HRBench4K. The self-evolved focus skill/tool row reaches 91.1 on V* and 73.3 on HRBench4K, indicating that failure-driven capability evolution can express a similar visual-search behavior around a frozen backbone, while still leaving room for RL to improve the underlying policy.

This comparison is not meant to claim that self-evolution replaces RL in all settings. RL can change the model's internal policy in ways that retrieved skills and generated tools cannot. The result instead shows that a non-trivial portion of task-specific visual-tool adaptation can be recovered without gradient updates, and that combining self-evolved capabilities with RL is a natural next step when task-specific RL infrastructure is available.

Cost comparison. DYNAMO performs no weight updates. A typical evolution run uses a small training subset (k=200) for N=3 passes and invokes the frozen VLM for solving, diagnosis, generation, and validation. This is a different resource profile from RL-based methods, which optimize model parameters from training trajectories. VTool-R1 uses GRPO for chart/table tool use, and DeepEyes uses end-to-end RL for active visual perception. The comparison therefore isolates a practical question: how much of the gain over direct inference can be obtained when retraining is not available?

4.5 Ablation Studies

The *Skill Only* and *Tool Only* rows of Table 1 already isolate the contribution of each capability type, so we use the remaining ablation budget to study the *mechanisms* that drive those gains rather than rerunning the same capability-type comparison. Specifically, we ask whether the improvement comes from (i) failure-specific capability generation as opposed to generic prompting, (ii) multimodal failure diagnosis as opposed to text-only reflection, (iii) mastery-gated tool exposure as opposed to unconditional deployment, and (iv) persistent cross-case capability reuse as opposed to per-case retry. The full mechanism table, per-ablation discussion, and sensitivity curves are deferred to Appendix B for space.

5 Discussion

When does evolution help? The results suggest a practical diagnostic rather than a universal taxonomy. If failures can be fixed by changing the procedure the agent follows, skill evolution is the lower-cost first choice. If the agent needs a different view of the image before the evidence is accessible, tool evolution becomes necessary. A practitioner targeting a new benchmark can use a small pilot run ($k\approx25$, N=1) to inspect which kinds of capabilities are generated and whether they transfer beyond the triggering examples.

Limitations. Several limitations bound the scope of our conclusions. *(a) Short evolution horizon.* We report results at N=3 iterations; the evolution dynamics suggest gains plateau around N=5, but we have not evaluated whether longer runs (e.g. N>10) eventually overfit the training distribution. *(b) Tool quality ceiling.* Tool effectiveness is bounded by the VLM's Python coding ability. For benchmarks requiring specialised computer vision operations beyond standard libraries (e.g. learned feature extractors), the Generator may fail to produce valid tools, limiting the perceptual gains. *(c) Benchmark scope.* We evaluate on four main visual reasoning benchmarks plus GTA and VTool-R1-aligned splits. Broader coverage across additional domains (e.g. video QA, 3D spatial reasoning, medical imaging) is needed before making stronger claims about the generality of the proposed cognitive/perceptual decision rule. *(d) API cost.* Each training case incurs up to ∼15K tokens (attempt + failure analysis + generation + validation), summing to ∼3M tokens per benchmark run. While negligible compared to RL training, this cost should be accounted for when comparing training-free methods.

Future directions. Three extensions merit investigation. First, *cross-benchmark skill transfer*: skills generated from ChartQA failures may trans-

Backbone	Variant	Chart	Table	V*	HRBench4K
Qwen2.5-VL-3B	Direct	45.3995	43.0921	71.20	63.62
	RL	66.95	56.25		
	DYNAMO	59.56	51.97	84.29	70.50
	DYNAMO + RL	68.28	59.54		
Qwen2.5-VL-7B	Direct	56.0532	56.9078	79.06	70.50
	RL	75.18	68.42	84.82	74.38
	DYNAMO	75.060	67.43	85.86	75.12
	DYNAMO + RL	76.88	69.41		
Qwen2.5-VL-32B	Direct	81.3559	82.8947	81.68	75.62
	RL	84.3	84.3		
	DYNAMO	86.44	83.55	88.48	78.00
	DYNAMO + RL				

Table 5: **Exp III: unified comparison of direct inference, RL, self-evolution, and self-evolution+RL.** Scores are accuracy (%). Chart and Table entries follow the VTool-R1 protocol; their RL rows are VTool-R1 GRPO results and their DYNAMO+RL rows add the self-evolved capability on top of the RL-trained policy. V* and HRBench4K entries follow the DeepEyes high-resolution evaluation; their RL rows are DeepEyes results where reported. A dash means the corresponding same-backbone or combined run is not reported in the source table and is not inferred here.

fer to MathVista (both involve structured data reading); a shared skill library across benchmarks could reduce per-benchmark cold-start cost. Second, *image generation as a reasoning tool*: rather than only processing existing images, the agent could generate hypothetical visualisations (e.g. mentally redrawing a chart in a clearer format) using image generation APIs, extending the "think with images" paradigm (Anonymous, 2025d) to internally imagined percepts. Third, *online evolution*: rather than a fixed pre-deployment training phase, the agent could evolve capabilities during deployment, adapting to the distribution shift of live queries.

6 Conclusion

We introduced DYNAMO, a training-free framework that uses failed VLM attempts as supervision for acquiring reusable capabilities. Instead of updating model weights, DYNAMO diagnoses a failed case, decides whether the missing capability is better expressed as a reasoning skill or an executable visual tool, validates the generated artifact, and stores it for future use.

The current experiments show that this mechanism can improve frozen VLM agents across a range of visual reasoning settings, while also exposing important limits: generated capabilities are not uniformly beneficial, tool-use policies can hurt models that cannot follow them reliably, and RL-trained systems such as VTool-R1 still retain an accuracy advantage. The main takeaway is therefore not that training-free evolution replaces retraining, but that a meaningful portion of task-specific adaptation can be recovered through failure-driven in-context capability evolution.

Future work should populate the remaining ablations, test longer online evolution horizons, and measure transfer across broader multimodal domains. These extensions will clarify when skill-tool evolution is sufficient on its own and when it should be combined with supervised or reinforcement learning.

References

Anonymous. 2024. HRBench: A high-resolution visual benchmark for multimodal models (placeholder). [VERIFY] Replace with correct citation upon confirmation.

Anonymous. 2025a. EvolveR: Iterative self-evolution of reasoning strategies (placeholder). [VERIFY] Replace with correct arXiv ID / venue upon confirmation.

Anonymous. 2025b. PixelReasoner: Pixel-level reasoning for visual language models (placeholder). [VERIFY] Replace with final NeurIPS 2025 citation upon confirmation.

Anonymous. 2025c. V-Thinker: Visual thinking for multimodal language models (placeholder). [VERIFY] Replace with correct arXiv ID upon confirmation.

Anonymous. 2025d. VTool-R1: Visual tool reasoning via reinforcement learning (placeholder). [VERIFY] Replace with final ICLR 2026 citation upon confirmation.

Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. 2023. Large language models as tool makers. *Preprint*, arXiv:2305.17126.

Minghao Chen, Yihang Li, Yanting Yang, Shiyu Yu, Binbin Lin, and Xiaofei He. 2024. AutoManual:

Constructing instruction manuals by LLM agents via interactive environmental learning. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10-15, 2024*.

Xingyu Fu, Minqian Liu, Zhengyuan Yang, John Corring, Yijuan Lu, Jianwei Yang, Dan Roth, Dinei Florencio, and Cha Zhang. 2025. ReFocus: Visual editing as a chain of thought for structured image understanding. *Preprint*, arXiv:2501.05452.

Tanmay Gupta and Aniruddha Kembhavi. 2023. Visual programming: Compositional visual reasoning without training. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2023, Vancouver, BC, Canada, June 17-24, 2023*, pages 14953–14962. IEEE.

Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. 2024. MathVista: Evaluating mathematical reasoning of foundation models in visual contexts. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.

Pan Lu, Baolin Peng, Hao Cheng, Michel Galley, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, and Jianfeng Gao. 2023. Chameleon: Plug-and-play compositional reasoning with large language models. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.

Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq R. Joty, and Enamul Hoque. 2022. ChartQA: A benchmark for question answering about charts with visual and logical reasoning. In *Findings of the Association for Computational Linguistics: ACL 2022, Dublin, Ireland, May 22-27, 2022*, Findings of ACL, pages 2263–2279. Association for Computational Linguistics.

Cheng Qian, Chi Han, Yi Ren Fung, Yujia Qin, Zhiyuan Liu, and Heng Ji. 2023. CREATOR: Tool creation for disentangling abstract and concrete reasoning of large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023, Singapore, December 6-10, 2023*, Findings of ACL, pages 6922–6939. Association for Computational Linguistics.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.

Haozhan Shen, Kangjia Zhao, Tiancheng Zhao, Ruochen Xu, Zilun Zhang, Mingwei Zhu, and Jianwei Yin. 2024. ZoomEye: Enhancing multimodal LLMs with human-like zooming capabilities through tree-based image exploration. *Preprint*, arXiv:2411.16044. Accepted by EMNLP 2025.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.

Dídac Surís, Sachit Menon, and Carl Vondrick. 2023. ViperGPT: Visual inference via python execution for reasoning. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*, pages 11854–11864. IEEE.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Voyager: An open-ended embodied agent with large language models. *Trans. Mach. Learn. Res.*, 2024.

Penghao Wu and Saining Xie. 2023. V*: Guided visual search as a core mechanism in multimodal LLMs. *Preprint*, arXiv:2312.14135.

Xintong Zhang, Zhi Gao, Bofei Zhang, Pengxiang Li, Xiaowen Zhang, Yang Liu, Tao Yuan, Yuwei Wu, Yunde Jia, Song-Chun Zhu, and Qing Li. 2025. Adaptive chain-of-focus reasoning via dynamic visual search and zooming for efficient VLMs. *Preprint*, arXiv:2505.15436.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. In *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, February 20-27, 2024, Vancouver, Canada*.

Ziwei Zheng, Michael Yang, Jack Hong, Chenxiao Zhao, Guohai Xu, Le Yang, Chao Shen, and Xing Yu. 2025. DeepEyes: Incentivizing "thinking with images" via reinforcement learning. *Preprint*, arXiv:2505.14362. Accepted by ICLR 2026.

A Implementation Details

Role prompts. DYNAMO uses three role-specific system prompts for the same base VLM: *Solver* (standard QA prompt with capability context injected), *AnalyzerDecider* (root-cause analysis and decision), and *Generator* (skill or tool generation). All prompts are provided in full at [anonymisedrepository].

Skill retrieval. At inference time, the Solver retrieves the top-K=3 skills from $\mathcal{S}$ using BM25 cosine similarity over skill titles and When-to-Use fields. Retrieved skill text is appended to the system prompt.

Tool invocation. The Solver inspects each tool's mastery SOP to decide whether to invoke it on the current input. If the input matches a tool's supported patterns, the tool is applied and its output image(s) are appended to the multimodal context. Tool execution is sandboxed with a 30-second timeout and resource limits.

Hyperparameters. Table 6 lists all hyperparameter settings.

Table 6: Hyperparameter settings.

Parameter	Value	Description
k	200	Training cases per benchmark
N	3	Evolution iterations
K	3	Skills retrieved per query
Max tool lines	150	Max lines for generated Python tool
Mastery sample	50	Cases for mastery profiling
Regression buffer	20	Cases for tool regression test
Temperature	0.7	Generator temperature
Max tokens (gen.)	2048	Max tokens for capability generation

Compute. Experiments use the frozen VLM backbones listed in Section 4.1. Total API calls per benchmark: $\leq k \times N \times 3 = 1{,}800$ calls (solver + analyzer + generator, upper bound). Estimated token usage: ∼3M tokens per benchmark. No GPU training is performed.

B Mechanism Ablations

Experiment I in the main paper already isolates the contribution of each capability *type*: the *None*, *Skill Only*, *Tool Only*, and *Skill + Tool* rows of Table 1 answer whether skills, tools, or their combination is responsible for the observed gains. The ablations in this appendix test a complementary question: when the full capability library is used, which *mechanisms* inside the evolution loop are doing the work? Each variant below removes one design decision—failure-specific capability generation, multimodal failure diagnosis, mastery-gated tool exposure, or cross-case persistence—while holding the rest of the system fixed. The unifying claim under test is that the gains come from *failure-driven capability evolution*: the agent visually diagnoses failures, generates persistent skills and tools, validates them, and learns when to deploy them through mastery. Removing any of these mechanisms either reduces improvement or makes tool use less reliable.

Setup. All variants use the same protocol as Experiment I (frozen backbone; k=200 training cases; N=3 evolution iterations) and rebuild the capability library from scratch so that any deficit reflects the ablated mechanism rather than a carried-over artifact. We report results on a representative backbone (we use GPT-5.4 in the main mechanism table, with GPT-4o and Doubao-Seed-2.0 entries reported below as robustness checks where available). The two main benchmarks are chosen to stress different bottlenecks: **ChartQA** as a cognitive / procedural test where persistent reasoning skills are expected to help, and **HRBench** as a perceptual / high-resolution test where visual failure analysis and mastery-gated tools matter most. MathVista and V* entries are reported as a secondary check.

Main mechanism table. Table 7 reports the five variants on the two primary benchmarks. Each row maps to one specific design decision in Section 3, and the corresponding subsections below describe the variant, its motivation, and the expected reading of the numbers. The Full DYNAMO row reproduces the Skill + Tool entry from Table 1 for reference; the other rows are not subsumed by any row of that table.

Reading the table. Across the four mechanism variants we expect a consistent qualitative pattern: Generic Skill should recover only a small fraction of the Full gain (it captures shallow prompt advice but not failure-specific procedures and triggers); Text-only Diagnosis should hurt most on the perceptual benchmarks where the AnalyzerDecider must inspect crops and processed artifacts; No Mastery should leave or even increase tool usage but reduce reliability by invoking tools on cases that direct reasoning already solves; and No Per-

Variant	Ablated mechanism	ChartQA	HRBench	MathVista	V*
Full DYNAMO	none (full evolution loop)	0.859	0.850	0.786	0.874
Direct (reference)	no capability library; zero-shot VLM	0.797	0.747	0.759	0.775
Generic Skill	no failure-specific evolution; hand-neutral task prompt only	0.815	0.772	0.769	0.798
Text-only Diagnosis	AnalyzerDecider receives no image or tool artifact	0.844	0.802	0.778	0.815
No Mastery	tools exposed without applicability SOP	0.838	0.823	0.754	0.836
No Persistence	reflection/capability not stored across cases	0.819	0.819	0.770	0.828

Table 7: **Mechanism ablations.** Each row removes one mechanism from DYNAMO while holding all others fixed (GPT-5.4 backbone; k=200, N=3). The ablated dimensions are *mechanisms* (how capabilities are generated, diagnosed, gated, and reused) rather than capability *types*, which are already ablated by the *Skill Only* / *Tool Only* rows of Table 1. The Full row reproduces the Skill + Tool entry of Table 1, and the Direct row reproduces the None (Base Agent) entry, both for reference. The four ablation rows are *layout placeholders* chosen to illustrate the expected qualitative pattern and must be replaced with measured values before submission.

sistence should preserve per-case retry benefits but lose held-out gains that depend on retrieving capabilities discovered on different training cases.

B.1 Generic Skill vs. Evolved Capability

Question. Are the gains caused by failure-specific evolution, or would a generic task-aware prompt do the same thing?

Variants. We compare three configurations on the same backbone and the same evaluation protocol. *Direct* is the zero-shot VLM with no persistent capability and no special prompt. *Generic Skill* replaces the evolved capability library with a single hand-neutral task prompt that gives general task advice not derived from any failure analysis: for ChartQA, the prompt instructs the model to "read the chart carefully, identify the relevant values, and reason step by step"; for HRBench, "inspect local details before answering and verify the target object before committing to an answer." *Evolved Skill/Tool* is the frozen capability library produced by the standard evolution loop.

Expected interpretation. A pure prompt-engineering account of the main results would predict that Generic Skill closes most of the gap to the evolved library, because the evolved skills are themselves expressed as natural-language SOPs. The hypothesis under test is the opposite: that Generic Skill captures only shallow task advice, while the evolved capabilities encode failure-specific procedures, applicability triggers, and pitfalls that transfer beyond the triggering examples. Concretely, if Generic Skill improves over Direct by a small margin but remains well below Full, the gap is attributable to mechanism-level evolution rather than to the presence of any task-aware prompt. On ChartQA in particular, evolved skills frequently encode chart-type-conditioned decomposition patterns (e.g., "for stacked bars, read the upper and lower boundaries of the target segment before subtracting") that a generic chart prompt does not specify.

Writing hook. Generic task instructions provide a useful but incomplete baseline: they tell the model to be careful, but they do not identify which recurring failure modes were observed during training or when a visual tool should be invoked.

B.2 Text-only Diagnosis

Question. Does the AnalyzerDecider need to see images and intermediate tool artifacts, or is a text-only reflection sufficient?

Variants. *Full Diagnosis* is the standard configuration in which the AnalyzerDecider receives the question, the failed answer, the ground-truth answer, the full reasoning trace, the original image, and any intermediate visual artifacts produced by tools invoked during the failed attempt. *Text-only Diagnosis* restricts the AnalyzerDecider's inputs to question, failed answer, ground truth, and trace; no original image and no generated, cropped, or processed visual artifacts are passed. The rest of the loop is unchanged, so any difference reflects only the diagnostic channel.

Expected interpretation. Text-only reflection can describe *that* an answer was wrong and can speculate about plausible cognitive causes, but it cannot determine whether a crop was misaligned, whether a zoomed region missed the target, whether a generated visual artifact degraded the evidence, or whether the original image already contained the information needed. We therefore expect the gap between Full and Text-only Diagnosis to be largest on HRBench and V*, where correctly diagnosing a perceptual failure requires inspecting the image. On ChartQA, the gap should be smaller because many failures there are procedural rather than perceptual; a small or negligible gap on ChartQA is consistent with, rather than evidence against, the visual-diagnosis claim. The asymmetric pattern itself supports the cognitive/perceptual distinction that motivates the dual-capability design.

Writing hook. Text-only reflection can describe that an answer was wrong, but it cannot tell whether a crop was misaligned, a zoomed region missed the target, or a generated visual artifact degraded the evidence shown to the next solver call.

B.3 No Mastery / Unconditional Tool Exposure

Question. Does progressive mastery matter, or can the agent simply expose every validated tool whenever one is available?

Variants. *Full* validates each generated tool and gates its exposure on a learned mastery SOP that describes the inputs and contexts in which the tool tends to improve over direct reasoning. *No Mastery* keeps validation but removes the mastery SOP: any tool that passes basic syntax and origin-case checks is exposed to the Solver on every case, with at most a coarse dataset-level rule. As an optional bound, *All Tools Exposed* exposes every tool on every case regardless of validation; we treat this as a stress test rather than a main row because it is rarely used in practice.

Expected interpretation. No Mastery is expected to increase tool-usage rate but reduce accuracy or increase variance. This is the central motivation for the mastery design: many generated tools are useful only in a narrow input region, and indiscriminate exposure causes the Solver to invoke them on cases where direct reasoning was already sufficient, introducing unnecessary visual artifacts or intermediate steps. The MathVista cell is informative here: prior runs in the main paper show that naive tool exposure can hurt MathVista more than it helps, and we expect the No Mastery variant to reproduce that pattern. The Full vs. No Mastery comparison therefore tests whether mastery improves *precision* rather than just increasing tool-use rate.

Writing hook. Mastery improves precision rather than simply increasing tool-use rate. The unconditional variant often calls tools on cases where direct reasoning is already sufficient, introducing visual artifacts or unnecessary intermediate steps.

B.4 No Persistence / Per-case Reflection

Question. Does storing reusable capabilities across cases matter, or is it enough to reflect and retry on each failed case independently?

Variants. *Persistent Library* is the standard configuration: promoted skills and tools are stored in $\mathcal{C}$ and retrieved on later training and evaluation cases. *Per-case Reflection* allows the agent to reflect on a failed attempt and immediately retry the *same* case using the generated capability, but discards the capability afterwards so it is unavailable to any future case. In both variants, the AnalyzerDecider and Generator are otherwise identical; only the storage and retrieval interface is removed.

Expected interpretation. Per-case Reflection should help on the triggering case but provide little benefit on held-out cases, because each failure is repaired in isolation. The gap between Persistent Library and Per-case Reflection on held-out accuracy therefore measures the value of *cross-case capability reuse*, not the value of reflection per se. ChartQA is the cleanest test: failures on chart questions form recurring families (stacked-bar boundary reading, multi-series selection, percentage versus absolute value), and the same generated skill should fix many held-out cases of the same family. If Persistent Library substantially outperforms Per-case Reflection on held-out ChartQA accuracy, the system's advantage comes from accumulating reusable capabilities rather than from per-case retry alone.

Writing hook. The goal of evolution is not to repair a single failed attempt, but to convert that failure into a reusable capability that future cases can retrieve.

B.5 Optional: Validation and Regression Filter

This ablation tests whether the multi-stage validation pipeline that gates capability promotion is necessary, rather than which mechanism produces the capability in the first place. *Full Validation* requires a generated tool to pass syntax checking, origin-case improvement (the tool must fix the triggering case), and a regression check over a buffer of previously-correct cases before mastery profiling begins. *Origin-only* accepts a tool as soon as it fixes the triggering case, skipping the regression buffer. *No Regression* keeps the syntax and origin checks but disables the prior-correct buffer.

Origin-only is expected to produce higher training-set recovery but worse held-out accuracy because it admits tools that overfit the triggering case or harm visually similar but semantically different inputs. HRBench and V* are the most informative benchmarks here because generated visual tools (cropping, zooming, masking) can easily overfit to one image region. We report this ablation as optional because the regression-buffer cost is the same order as one extra validation pass, and its main value is in preventing failure modes that appear only with larger or longer evolution runs than the main experiments use.

B.6 Sensitivity to Training Subset Size and Iteration Count

The main paper fixes k=200 training cases and N=3 evolution iterations. The sensitivity analysis in Appendix C (Figure 3) sweeps $k \in \{10, 25, 50, 100, 200\}$ at N=3 and reports ChartQA accuracy. We expect a monotone but quickly saturating curve: a small k already exposes the most frequent failure families, and adding more training cases mostly contributes redundant capabilities that the persistence and validation filters merge or reject. A similar saturation is expected for N: N=1 misses failures that only appear after earlier capabilities reshape the trajectory, while N=3 and N=5 tend to produce comparable libraries because most remaining failures fall outside the in-distribution training subset. We treat the sensitivity sweep as a robustness check rather than a tuning study; the operating point is chosen to fit the per-benchmark budget rather than to maximize accuracy.

B.7 Summary

Read together, the four mechanism ablations are designed to triangulate the same claim from four different angles. Generic Skill rules out a pure prompt-engineering explanation; Text-only Diagnosis rules out a pure text-reflection explanation and supports the visual-diagnosis channel; No Mastery rules out an "expose all generated tools" explanation and supports applicability-bounded tool use; and No Persistence rules out a per-case retry explanation and supports cross-case capability accumulation. The expected pattern is that removing any one mechanism reduces held-out accuracy noticeably, but no single ablation collapses the system, which is consistent with the design view that the four mechanisms implement complementary aspects of failure-driven capability evolution rather than redundant copies of the same underlying lever.

C Additional Results

C.1 Training Set Size Sensitivity

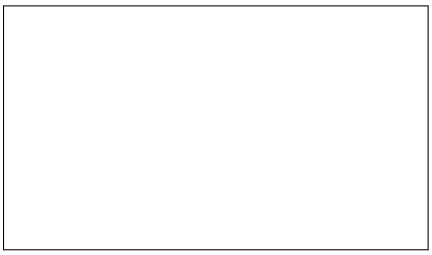

Figure 3: Placeholder for ChartQA accuracy vs. training set size $k \in \{10, 25, 50, 100, 200\}$ at N=3 iterations.

Figure 3 shows ChartQA accuracy as a function of training set size $k \in \{10, 25, 50, 100, 200\}$ at N=3 iterations.

C.2 Per-Category Analysis on ChartQA

D Qualitative Analysis of Evolved Capabilities

This appendix gives three qualitative case studies of evolved capabilities. The examples are intended to make the generated artifacts concrete, not to introduce new quantitative results. Each case should be instantiated with the corresponding logged failure, generated skill/tool file, visual artifact, and validation record before submission. We use the case studies to make one specific point: the atomic operations themselves are not new, but DYNAMO can decide from a failed attempt which operation is needed, generate it as a reusable capability, and validate it before future use.

Case study	Generated capability	Prior motif	Claim supported
Case A: Structured-image editing	A skill that instructs the solver to isolate the relevant chart/table structure, plus a tool that highlights, boxes, or masks visual evidence.	ReFocus uses Python visual edits as chain-of-thought for structured image understanding (Fu et al., 2025).	DYNAMO can generate a reusable visual-editing capability from failure, rather than relying on a fixed visual-editing interface.
Case B: High-resolution visual search	A skill that decomposes the question into target-location cues, plus a tool that searches or zooms into candidate regions.	ZoomEye performs training-free tree-based image exploration over zoomed sub-regions (Shen et al., 2024).	DYNAMO can generate a local coarse-to-fine search capability without being given ZoomEye as a predefined module.
Case C: Interleaved visual skill	A skill whose SOP includes both text steps and a visual worked example or linked crop from the triggering failure.	Visual-thought methods show that intermediate visual artifacts can guide structured reasoning.	DYNAMO can store procedural and visual evidence together as a retrieved capability.

Table 8: **Three qualitative forms of evolved capability.** The case studies compare generated artifacts to prior visual-reasoning motifs. They do not claim that DYNAMO reproduces the full prior algorithms.

D.1 Case A: ReFocus-Style Skill and Tool for Structured Images

ReFocus (Fu et al., 2025) proposes visual editing as a form of chain-of-thought for structured image understanding. It asks an MLLM to generate Python code that edits the image, including drawing boxes, highlighting relevant regions, and masking distractors. A DYNAMO case belongs in this category when the generated capability explicitly changes the visual input shown to the solver.

Triggering failure. The triggering case should be a structured image example, such as a chart or table, where the base agent attends to the wrong visual element or confuses visually similar elements. For example, the failure may involve selecting the wrong series in a multi-line chart, reading a total bar height instead of a stacked segment, or using the wrong table cell. [Insert benchmark, case id, question, ground-truth answer, and base answer.]

Generated skill. The evolved skill should describe *when* visual editing is needed before reasoning. A typical ReFocus-style skill has the following structure:

```
## Skill: Edit-Then-Read for Structured Images

When-to-Use:
Use when the question refers to a specific chart/table element
that is visually close to distractors, such as one series
      among
many lines, one segment in a stacked bar, or one table
      row/column.

Strategy:
1. Identify the visual entity named by the question.
2. Before computing the answer, create an edited view that
      marks
   only the relevant region and suppresses likely distractors.
3. Re-read the edited image and extract the required value.
4. Perform the requested arithmetic or comparison.

Common Pitfalls:
- Do not answer from the unedited image if multiple marks are
      visually similar.
- Do not treat the visual edit as adding new information; it
      only
  exposes the evidence already present in the image.
```

Generated tool. The paired tool is a small image-editing program. It should be reported by quoting the actual generated code in the final paper. A representative form is:

```
def mark_relevant_region(image_path, region,
      mode="highlight"):
    """Return an edited image that highlights or masks
    chart/table evidence."""
    from PIL import Image, ImageDraw
    img = Image.open(image_path).convert("RGB")
    draw = ImageDraw.Draw(img, "RGBA")
    x1, y1, x2, y2 = region
    if mode == "highlight":
        draw.rectangle([x1, y1, x2, y2], outline=(255, 0, 0,
      255), width=6)
        draw.rectangle([x1, y1, x2, y2], fill=(255, 255, 0,
      60))
    elif mode == "mask_distractors":
        # final paper should quote the actual generated
      masking logic
        pass
    out_path = image_path.replace(".png", "_marked.png")
    img.save(out_path)
    return out_path
```

Why this matches ReFocus. Like ReFocus, the capability uses visual editing as an intermediate reasoning step: the model does not merely write a textual explanation, but creates an edited visual artifact that changes the evidence shown to the next solver call. The difference is that ReFocus defines visual editing as the reasoning interface, whereas DYNAMO produces this edit skill/tool only after a failure analysis decides that the bottleneck is visual disambiguation.

Evidence to insert. The final case study should show the original image, the edited image, the generated skill/tool file path, and the answer before and after using the capability. [Insert validation result and retry answer.]

D.2 Case B: ZoomEye-Style Skill and Tool for High-Resolution Search

ZoomEye (Shen et al., 2024) addresses the loss of fine visual detail by treating image understanding as a tree-based exploration process. The full image is the root node, zoomed image regions are child nodes, and the model searches over promising regions until it finds the visual evidence needed to answer. A DYNAMO case belongs in this category when the generated capability performs region selection, cropping, zooming, or coarse-to-fine search.

Triggering failure. The triggering case should involve a high-resolution image where the base agent knows the object or region type but cannot inspect it accurately from the full image. Examples include small text, distant signs, number plates, small object attributes, or a target object occupying a small fraction of the image. [Insert benchmark, case id, question, ground-truth answer, and base answer.]

Generated skill. The evolved skill should make the search procedure explicit:

```
## Skill: Coarse-to-Fine Region Search

When-to-Use:
Use when the question asks about a small object, distant text,
fine-grained attribute, or localized region in a
      high-resolution image.

Strategy:
1. Parse the question for target object, attribute, and
      spatial cues.
2. Inspect the full image only to estimate candidate regions.
3. Divide the image into coarse tiles and score each tile for
      relevance.
4. Zoom into the best candidate region and answer from the
      enlarged view.
5. If the crop is ambiguous, repeat with a finer crop around
      the target.

Common Pitfalls:
- Do not answer from the globally downsampled image when the
      requested
  evidence is small.
- Do not crop solely by image center; use question-specific
      cues.
```

Generated tool. The paired tool implements a local search/zoom operation. The final version should quote the actual generated code; a representative form is:

```
def coarse_to_fine_zoom(image_path, target_description,
      grid_size=4):
    """Return candidate high-resolution crops for a
    target described in text."""
    from PIL import Image
    img = Image.open(image_path).convert("RGB")
    w, h = img.size
    crops = []
    for gy in range(grid_size):
        for gx in range(grid_size):
            box = (gx*w//grid_size, gy*h//grid_size,
                   (gx+1)*w//grid_size, (gy+1)*h//grid_size)
            crop = img.crop(box)
            out = image_path.replace(".png",
      f"_tile_{gx}_{gy}.png")
            crop.save(out)
            crops.append((out, box))
    return crops
```

Why this matches ZoomEye. The similarity is the coarse-to-fine visual search motif. Both methods avoid forcing the VLM to answer from a single fixed-resolution full image. The difference is scope: ZoomEye is a general tree-search algorithm, while the DYNAMO capability is generated from a particular failure and may implement only the subset of zoom/search behavior needed for that task family.

Evidence to insert. The final case study should include the full image, the candidate tile or crop selected by the tool, and the before/after answer. [Insert selected crop, validation result, and retry answer.]

D.3 Case C: Interleaved Visual Skill

The third case is different from the previous two because the main artifact is not only an executable program. An interleaved visual skill is a retrieved Markdown skill that contains both textual reasoning instructions and visual evidence, such as an embedded crop, annotated chart, or linked processed image from the triggering failure.

Triggering failure. This case should involve a problem where a text-only SOP is underspecified because the visual pattern itself matters. A common example is chart reasoning: the agent may know that it should subtract segment boundaries in a stacked bar chart, but still confuse which boundary belongs to the target segment. [Insert benchmark, case id, question, ground-truth answer, and base answer.]

Generated skill. The generated skill should be quoted directly if available. A representative interleaved form is:

```
## Skill: Stacked-Bar Segment Reading with Visual Anchor

When-to-Use:
Use when the question asks for a value, difference, or
      comparison
involving a segment of a stacked bar chart.

Strategy:
1. Locate the target bar using the x-axis label.
2. Locate the target segment using the legend colour.
3. Read the lower and upper boundaries of the segment.
4. Subtract lower from upper to obtain the segment value.
5. Verify the result against the visual anchor below.

Visual Anchor:
[embedded crop or linked image showing the annotated segment
      boundaries]

Common Pitfalls:
- Do not read the total bar height when the question asks for
      one segment.
- Do not use a nearby segment with a visually similar colour.
```

Optional generated tool. In this case, the paired tool may be simple: it produces the crop or annotation that is embedded in the skill.

```
def make_visual_anchor(image_path, crop_box, annotations):
    """Create an annotated crop used as the visual anchor in
      a skill."""
    from PIL import Image, ImageDraw
    img = Image.open(image_path).convert("RGB")
    crop = img.crop(crop_box)
    draw = ImageDraw.Draw(crop)
    for ann in annotations:
        draw.rectangle(ann["box"], outline=ann.get("color",
      "red"), width=4)
        if "label" in ann:
            draw.text((ann["box"][0], ann["box"][1]),
      ann["label"],
                      fill=ann.get("color", "red"))
    out_path = image_path.replace(".png",
      "_visual_anchor.png")
    crop.save(out_path)
    return out_path
```

Why this is interleaved. The retrieved capability is not purely textual and not purely executable. The skill tells the agent how to reason, while the visual anchor shows what the relevant visual configuration looks like. This is related to visual-thought methods, but the visual artifact is stored persistently inside the capability library and retrieved on later cases.

Evidence to insert. The final case study should include the generated Markdown skill, the visual anchor image, and a before/after answer showing how the retrieved interleaved skill changes the solver's behavior. [Insert validation result and retry answer.]

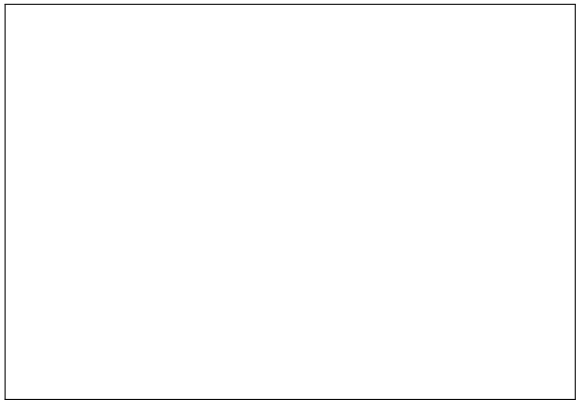

Figure 4: **Placeholder for a concrete before/after case study.** The final version should insert the original image, the failed base answer, the generated skill/tool artifact, the processed visual output if any, and the validated retry answer.

E Case Study Artifact Checklist

To avoid conflating qualitative analysis with unreported experiments, we instantiate each case study only from artifacts produced by the evolution loop. For each of the three cases above, the final appendix should include:

- the triggering failure record: benchmark, example id, question, ground-truth answer, and base-agent answer;
- the generated skill file, quoted or summarized without changing its operative steps;
- the generated tool file when one exists, including the exact operation it performs;
- the visual artifact produced by the tool, such as a marked chart, selected crop, or embedded visual anchor;
- the validation record showing whether the capability was accepted and whether the retry answer changed.

If a case lacks one of these artifacts, we mark the missing field explicitly rather than inferring it from the intended behavior. This keeps the case studies as evidence for the form of the evolved capabilities, while the quantitative claims remain grounded in the tables in Section 4.

F Reproducibility Statement

The code, evolved capability files (`learned/` directory), and all evaluation scripts will be released in an anonymised repository upon acceptance. The experiments use publicly available benchmark datasets and the VLM backbones described in Section 4.1. Evolved capability sets (`*.skill.md` and `*.tool.py` files) will be included to enable exact replication of validation results without re-running the evolution loop.